



EAST WEST UNIVERSITY

REPORT

Department of Computer Science and Engineering

Course Code: CSE207 Course Title:Data Structure

Section: 4 Semester: Summer 2026

Submitted By

Nabila Najnin(2022-2-60-119)

Wasir Abu Labib(2025-2-60-333)

Submitted to

Dr. Md. Atiqur Rahman

Assistant Professor

Department of Computer Science and Engineering

Date of Submission: 10.07.2026

Table of Contents

Table of Contents.....	1
1. Introduction.....	3
2. Objectives	3
3. System Overview	3
4. System Modules.....	4
4.1 Customer Module	4
4.2 Kitchen Module.....	5
4.3 Admin Module	6
5. Data Structures Used.....	6
5.1 Structures.....	6
5.2 Linked List.....	7
5.3 Queue.....	8
5.4 Stack	10
6. Algorithms Used.....	10
6.1 Bubble Sort.....	10
6.2 Sequential Search.....	10
6.3 Recursion.....	11
7. Program Flow.....	11
8. Complexity Analysis	11
9. Input Validation	12
10. Memory Management.....	12
11. Advantages of the System	13
12. Limitations	13
13. Future Improvements.....	13
14. Conclusion	14

1. Introduction

The Restaurant Management System is a console-based application developed in the C programming language as part of the Data Structure (CSE207) course. The system was designed to organise the day-to-day activities of a small restaurant, including order placement, kitchen preparation, and menu administration, within a single, menu-driven program.

The application follows a modular, menu-driven design that allows a user to move between distinct areas of functionality without difficulty. Each order is tracked through a defined lifecycle: it is created and stored while active, forwarded to the kitchen for preparation, and finally transferred to either a completed or a cancelled order history. This lifecycle mirrors the way orders are handled in an actual restaurant setting.

A central aim of the project was to demonstrate how several data structures can be combined within one program to solve a practical problem. Linked lists are used to manage active orders, a queue maintains the sequence in which orders are prepared, and stacks are used to record completed and cancelled orders. The Bubble Sort algorithm is applied to arrange menu items by price.

Developing this system provided practical experience in combining data structures with modular functions to address a real-world scenario. It also reinforced our understanding of dynamic memory allocation, input validation, and the overall discipline of structured program design.

2. Objectives

The principal objective of this project was to design and implement a simple Restaurant Management System in C that simulates the core operations of a restaurant, while applying the programming concepts covered in the Structured Programming course. The specific objectives are as follows:

- To develop a menu-driven restaurant management application.
- To allow customers to view the food menu and place orders.
- To manage restaurant orders using appropriate data structures.
- To process customer orders in the sequence in which they are received.
- To enable administrators to add, update, delete, and display menu items.
- To calculate the total bill automatically for each order.
- To generate reports covering completed, cancelled, and active orders.
- To strengthen programming skills through the use of structures, linked lists, queues, stacks, sorting, and functions.

3. System Overview

The Restaurant Management System is organised into three principal modules, each responsible for a distinct stage of the restaurant workflow: the Customer Module, the Kitchen Module, and the Admin Module. Together, these modules cover the complete cycle of a restaurant order, from placement through preparation to administrative reporting.

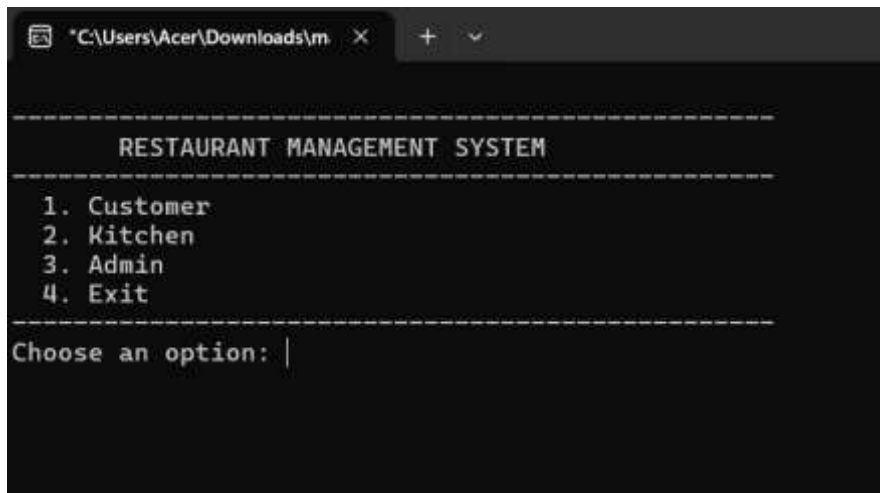


Figure 1: Main menu of the Restaurant Management System

4. System Modules

4.1 Customer Module

The Customer Module forms the primary point of interaction between the restaurant and its customers. It provides a menu-driven interface through which a customer can view the food menu, place an order, search for an existing order, and cancel an order that has not yet been prepared.

When placing an order, the system first records the customer's name and then allows one or more items to be selected by entering the corresponding item ID and quantity. Each item ID is validated against the current menu; an invalid entry produces an error message and prompts the customer to try again. Quantities are similarly validated, and only positive values are accepted, which prevents incomplete or malformed orders from being stored.

Once all items have been selected, the system calculates the total bill automatically and assigns a unique order ID. The order is then added to the active order list and placed in the kitchen queue for preparation. Customers may retrieve the details of an active order, including the items ordered, the total bill, and the current status, by searching with the order ID. Should a customer choose not to proceed, the cancel option removes the order from both the active order list and the kitchen queue and transfers it to the cancelled order history.



Figure 2 (left): Customer menu

ID	ITEM	PRICE
1	Chicken Burger	5.50
2	Beef Steak	12.75
3	Cheese Pizza	8.00
4	Veg Salad	3.25
5	Cold Coffee	2.50
6	Vanilla Ice Cream	2.00
6 item(s)		

Figure 3 (right): Restaurant food menu

ID	ITEM	PRICE
1	Chicken Burger	5.50
2	Beef Steak	12.75
3	Cheese Pizza	8.00
4	Veg Salad	3.25
5	Cold Coffee	2.50
6	Vanilla Ice Cream	2.00

6 item(s)
Item ID (0 to finish): 1
Quantity: 1
> Added 1 x Chicken Burger.
Add another item? (y/n): y

ID	ITEM	PRICE
1	Chicken Burger	5.50
2	Beef Steak	12.75
3	Cheese Pizza	8.00
4	Veg Salad	3.25
5	Cold Coffee	2.50
6	Vanilla Ice Cream	2.00

6 item(s)
Item ID (0 to finish): 2
Quantity: 2
> Added 2 x Beef Steak.

Figure 4a: Order placement -item selection

```
> Order placed and sent to the kitchen.
-----
Order #1001    Customer: ariyana    [Waiting]
-----
Chicken Burger          1 x      5.50 =      5.50
Beef Steak              2 x     12.75 =     25.50
TOTAL                   31.00
```

Figure 4b: Order placement- bill summary

4.2 Kitchen Module

The Kitchen Module manages the preparation of orders once they have been placed. Every new order is automatically appended to the kitchen queue, which operates on a First-In-First-Out (FIFO) basis so that orders are prepared strictly in the sequence in which they were received.

This module offers two options. The View Waiting Orders option reports the total number of orders awaiting preparation and displays the next order in line, together with its order number and customer name, allowing kitchen staff to see at a glance what should be prepared next. The Prepare Next Order option removes the order at the front of the queue, updates its status from Waiting to Completed, transfers it from the active order list to the completed order history, and displays its full details on screen. This continues until the queue is empty, ensuring that the kitchen workflow remains fair and efficient.

KITCHEN MENU	

1. View Waiting Orders	
2. Prepare Next Order	
3. Back	

Figure 5 (left): Kitchen menu

> Order #1001 prepared and served.				

Order #1001	Customer: ariyana	[Completed]		

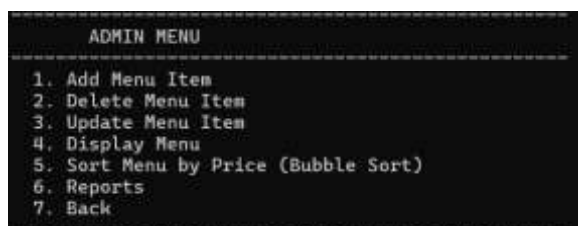
Chicken Burger	1 x	5.50 =	5.50	
Beef Steak	2 x	12.75 =	25.50	
TOTAL			31.00	

Figure 6 (right): Preparing the next order

4.3 Admin Module

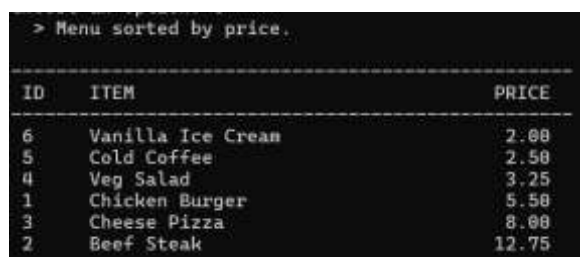
The Admin Module gives the administrator control over the restaurant menu and access to the reports generated by the system. Unlike the Customer Module, it permits modification of menu data and provides a consolidated view of restaurant activity.

Through this module, the administrator can add a new menu item by supplying its name and price, with a unique item ID assigned automatically; delete a menu item that is no longer offered; update the name or price of an existing item after the system locates it by ID; and display the full menu together with item IDs and prices for verification. A dedicated sorting option arranges menu items by price in ascending order using the Bubble Sort algorithm, which makes prices easier to compare and improves the overall presentation of the menu. The module also provides access to reports on completed orders, cancelled orders, total sales, and the total number of orders processed.



ADMIN MENU	
1.	Add Menu Item
2.	Delete Menu Item
3.	Update Menu Item
4.	Display Menu
5.	Sort Menu by Price (Bubble Sort)
6.	Reports
7.	Back

Figure 7 (left): Admin menu



> Menu sorted by price.		
ID	ITEM	PRICE
6	Vanilla Ice Cream	2.00
5	Cold Coffee	2.50
4	Veg Salad	3.25
1	Chicken Burger	5.50
3	Cheese Pizza	8.00
2	Beef Steak	12.75

Figure 8 (right): Menu sorted by Bubble Sort

5. Data Structures Used

Several data structures were combined in this project to organise restaurant data efficiently, with each structure suited to the particular task it performs.

5.1 Structures

The C struct construct is used throughout the project to group related information into single units. The main structures are MenuItem, OrderLine, Order, QNode, and SNode. MenuItem stores the ID, name, and price of a food item; OrderLine records the name, quantity, and unit price of each item selected within an order; and Order holds the complete details of a customer order, including the customer name, order ID, list of ordered items, total bill, current status, and a pointer to the next order in the active order list. QNode represents a node in the kitchen queue, while SNode represents a node in the completed and cancelled order stacks. Grouping related fields in this way keeps the program organised and makes the code considerably easier to maintain.

```

struct MenuItem
{
    int id;
    char name[NAME_LEN];
    float price;
};

struct OrderLine
{
    int itemId;
    char itemName[NAME_LEN];
    int quantity;
    float unitPrice;
};

struct Order
{
    int orderId;
    char customerName[NAME_LEN];
    struct OrderLine lines[MAX_LINES];
    int lineCount;
    float totalBill;
    char status[STATUS_LEN];
    struct Order *next;
};

struct QNode //used for kitchen queue
{
    struct Order *order;
    struct QNode *next;
};

struct SNode //used for storing the completed/cancelled order using stack
{
    struct Order *order;
    struct SNode *next;
};

```

Figure 9: Structures used in the program

5.2 Linked List

A singly linked list is used to store all active customer orders. When an order is placed, a new node is created and appended to the end of the list, where it remains until it is either completed or cancelled and subsequently removed. The linked list is also searched whenever a customer or the kitchen needs to locate an order by its unique ID. Because nodes can be inserted or removed without shifting other elements in memory, the linked list is well suited to a collection whose size changes frequently. The principal operations implemented are insert active order, search order, and remove active order.

```

void insertActiveOrder(struct Order *order)
{
    struct Order *current;

    order->next = NULL;
    if (activeHead == NULL)
    {
        activeHead = order;
        return;
    }
    current = activeHead;
    while (current->next != NULL)
    {
        current = current->next;
    }
    current->next = order;
}

```

Figure 10a: Linked list -insertion

```

struct Order *removeActiveOrder(int orderId)
{
    struct Order *current = activeHead;
    struct Order *previous = NULL;

    while (current != NULL && current->orderId != orderId)
    {
        previous = current;
        current = current->next;
    }
    if (current == NULL)
    {
        return NULL;
    }
    if (previous == NULL)
    {
        activeHead = current->next;
    }
    else
    {
        previous->next = current->next;
    }
    current->next = NULL;
    return current;
}

```

Figure 10b: Linked list -search

```

void printOrder(const struct Order *order)
{
    int i;

    printRule();
    printf(" Order #%d Customer: %s [%s]\n",
        order->orderId, order->customerName, order->status);
    printRule();
    for (i = 0; i < order->lineCount; i++)
    {
        printf(" %25s %3d x %8.2f = %8.2f\n",
            order->lines[i].itemName,
            order->lines[i].quantity,
            order->lines[i].unitPrice,
            order->lines[i].quantity * order->lines[i].unitPrice);
    }
    printf(" %25s %25.2f\n", "TOTAL", order->totalBill);
    printRule();
}

```

Figure 10c: Linked list - removal

5.3 Queue

A queue enforces the First-In-First-Out discipline required by the kitchen. Each new order is inserted at the rear of the queue, and the order at the front is removed when the kitchen begins preparing it, guaranteeing that customers are served in the order in which they placed their requests. The operations implemented for this structure are enqueue order, dequeue order, and remove order from queue, the last of which supports cancellation of an order that has not yet reached the front of the queue.

```

void enqueueOrder(struct Order *order)
{
    struct QNode *node = (struct QNode *)malloc(sizeof(struct QNode));

    if (node == NULL)
    {
        printf(" | Out of memory - order not queued.\n");
        return;
    }
    node->order = order;
    node->next = NULL;

    if (queueRear == NULL)
    {
        queueFront = node;
        queueRear = node;
    }
    else
    {
        queueRear->next = node;
        queueRear = node;
    }
}

```

Figure 11a: Queue — enqueue

```

struct Order *dequeueOrder(void)
{
    struct QNode *node;
    struct Order *order;

    if (queueFront == NULL)
    {
        return NULL;
    }
    node = queueFront;
    order = node->order;

    queueFront = node->next;
    if (queueFront == NULL)
    {
        queueRear = NULL;
    }
    free(node);
    return order;
}

```

Figure 11b: Queue — dequeue

```

int removeFromQueue(int orderId)
{
    struct QNode *current = queueFront;
    struct QNode *previous = NULL;

    while (current != NULL && current->order->orderId != orderId)
    {
        previous = current;
        current = current->next;
    }
    if (current == NULL)
    {
        return 0;
    }
    if (previous == NULL)
    {
        queueFront = current->next;
    }
    else
    {
        previous->next = current->next;
    }
    if (current == queueRear)
    {
        queueRear = previous;
    }
    free(current);
    return 1;
}

```

Figure 11c: Queue — removal

5.4 Stack

Two stacks are used to record the history of the restaurant's orders: one for completed orders and one for cancelled orders. Each order is pushed onto the relevant stack at the point it is completed or cancelled. Because a stack follows the Last-In-First-Out principle, the most recently completed or cancelled order is always the first to appear when a report is generated, which makes it straightforward to review the restaurant's most recent activity.

```
void pushCompleted(struct Order *order)
{
    strcpy(order->status, "Completed");
    pushStack(&completedTop, order);
}

void pushCancelled(struct Order *order)
{
    strcpy(order->status, "Cancelled");
    pushStack(&cancelledTop, order);
}
```

Figure 12: Stack operations

6. Algorithms Used

Three algorithmic techniques underpin the core functionality of the system.

6.1 Bubble Sort

Bubble Sort is used in the Admin Module to arrange menu items by price. The algorithm repeatedly compares adjacent items and swaps them if the first item's price exceeds the second, continuing this process until the entire menu is sorted in ascending order. Its simplicity makes it well suited to a menu of the modest size handled by this project.

```
void bubbleSortMenu(void)
{
    int i, j, swapped;
    for (i = 0; i < menuCount - 1; i++)
    {
        swapped = 0;
        for (j = 0; j < menuCount - 1 - i; j++)
        {
            if (menu[j].price > menu[j + 1].price)
            {
                swapMenuItems(j, j + 1);
                swapped = 1;
            }
        }
        if (!swapped)
        {
            break;
        }
    }
}
```

Figure 13: Bubble Sort function

6.2 Sequential Search

Sequential search is used whenever a menu item or an order needs to be located, for example when searching by item ID or order ID. The program examines each record in turn until the required entry

is found or the list is exhausted. Given the relatively small volume of data handled by the system, this approach is both simple and effective.

6.3 Recursion

Recursion is applied in the `countStack()` function, which counts the number of completed or cancelled orders by having the function call itself until the end of the stack is reached, rather than relying on an explicit loop. This provided practical experience with a further core technique introduced in the Structured Programming course.

7. Program Flow

Execution begins in the `main()` function, which first loads the default menu items into the system before displaying the main menu. From here, the user selects one of four options: Customer, Kitchen, Admin, or Exit. Selecting Customer, Kitchen, or Admin transfers control to the corresponding module, where food orders are placed, waiting orders are prepared, or menu items and reports are managed, respectively. When the user selects Exit, all dynamically allocated memory is released before the program terminates, which keeps the program's resource usage clean and prevents memory leaks.

8. Complexity Analysis

An efficient program is not only correct but also economical in the time and memory it consumes. Table 1 analyses the time complexity of the principal operations performed on each data structure used in the system, where n denotes the number of active orders, waiting orders, or menu items, as appropriate to the operation.

Data Structure	Operation	Time Complexity	Remarks
Linked List (active orders)	Insert (append)	$O(n)$	Traverses to the tail; could be reduced to $O(1)$ with a tail pointer
Linked List (active orders)	Search by order ID	$O(n)$	Sequential search through the list
Linked List (active orders)	Delete	$O(n)$	Requires locating the node before unlinking it
Queue (kitchen)	Enqueue	$O(1)$	Insertion at the rear using a tail pointer
Queue (kitchen)	Dequeue	$O(1)$	Removal from the front
Queue (kitchen)	Remove by order ID	$O(n)$	Used only when an order is cancelled before preparation
Stack (completed / cancelled)	Push	$O(1)$	Insertion at the top of the stack

Data Structure	Operation	Time Complexity	Remarks
Stack (completed / cancelled)	Count (recursive)	$O(n)$	One recursive call per stack node
Menu Array	Sequential search by ID	$O(n)$	Used for lookup during ordering and administration
Menu Array	Bubble Sort by price	$O(n^2)$	Acceptable given the modest size of the menu

Table 1: Time complexity of the principal operations

Every structure used in the system therefore offers at least one operation with constant-time performance in its intended role: the queue provides $O(1)$ enqueue and dequeue for kitchen processing, and the stack provides $O(1)$ push for recording order history. The $O(n)$ and $O(n^2)$ operations arise chiefly from the sequential search and Bubble Sort strategies, which were adopted deliberately for their simplicity given the small volume of data — typically a handful of menu items and a modest number of concurrent orders — that a single-restaurant system of this scope is expected to handle.

9. Input Validation

Input validation is applied throughout the program to reduce the likelihood of user error and to keep the stored data consistent. The main validation rules implemented are:

- Customer name cannot be left empty.
- Menu price must be greater than zero.
- Order quantity must fall between 1 and 100.
- Invalid menu IDs are rejected with an error message.
- Invalid order IDs produce an appropriate notification.
- Menu selections outside the valid range are not accepted.
- Integer and floating-point inputs are checked before use.

These checks reduce the likelihood of incorrect data being stored and contribute to the overall reliability of the program.

10. Memory Management

The system relies on dynamic memory allocation to create orders, queue nodes, and stack nodes as they are needed, using `malloc()` at each point of creation. Before the program exits, all allocated memory is released by the `freeAll()` function, which frees the active order list, the queue nodes, and both the completed and cancelled order stacks. This disciplined approach to allocation and deallocation helps prevent memory leaks and keeps the program efficient throughout execution.

11. Advantages of the System

- Simple and user-friendly interface.
- Straightforward placement and management of customer orders.
- Automatic calculation of the total bill.
- Fair, sequence-based order processing through the kitchen queue.
- Organised storage of completed and cancelled orders.
- Convenient administration of menu items.
- Generation of useful reports for the administrator.
- Integration of multiple data structures within a single application.
- A clear demonstration of modular programming practice.

12. Limitations

Although the system performs its intended tasks successfully, it is subject to a number of limitations:

- Data is not saved permanently once the program is closed.
- No login system exists for customers or administrators.
- The menu has a fixed maximum size.
- Orders cannot be modified once they have been placed.
- The system operates only through a console interface.
- Only a single user can operate the program at any one time.

These limitations are reasonable given that the project was developed as a course assignment within Structured Programming.

13. Future Improvements

A number of extensions could be pursued to develop the project further:

- Persist restaurant data using files or a database.
- Introduce separate login panels for customers and administrators.
- Develop a graphical user interface (GUI).
- Generate printable customer bills.
- Add online payment options.
- Include a table reservation facility.
- Provide daily and monthly sales reports.
- Allow customers to update their orders prior to preparation.
- Adopt more efficient searching algorithms for menu lookup.

These improvements would move the system closer to what would be required in an actual restaurant environment.

14. Conclusion

The Restaurant Management System is a console-based application, developed in the C programming language, that demonstrates how core programming concepts can be applied to a practical problem. Through its three modules, the system manages menu items, customer orders, kitchen processing, and reporting in a coherent and organised manner.

The project draws on a range of data structures, including structures, linked lists, queues, and stacks, to organise data efficiently, and applies Bubble Sort to arrange menu items by price and recursion to count completed and cancelled orders. Together, these elements illustrate how structured design and appropriate data structures can be combined to build a functioning application from modest components.

Working on this project deepened our understanding of modular programming, dynamic memory allocation, input validation, and the practical use of data structures, and gave us hands-on experience in designing a menu-driven application with several interacting modules. While the system has the limitations noted above, it meets the objectives set out for the project and provides a reasonable foundation for further development.